

Month 1 · Day 1

Searching, Sorting, and the Power of Order

Two-hour instructional module · GATE + interview practice · C++17

Central question: What information does ordering give us, and what does it cost to create or preserve that order?

Session map

Time	Activity	Main idea
0–15 min	Motivation and specification	A precise problem comes before an algorithm
15–35 min	Brute force	Establish correctness and expose repeated work
35–60 min	Sort + binary search	Ordering creates a searchable structure
60–90 min	Sort + two pointers	Monotonicity lets us eliminate possibilities
90–110 min	Unique-pair variation	Duplicates test whether the invariant is understood
110–120 min	Synthesis and exit ticket	Choose methods from constraints, not keywords

Learning objectives

By the end of this lesson, you should be able to:

1. Write a precise specification for a pair-search problem.
2. Compare brute force, sorting, binary search, two pointers, and hashing.
3. Explain what sorting makes easier and what information it may destroy.
4. State and use a two-pointer invariant.
5. Preserve original indices while sorting values.
6. Handle duplicates, self-pairing, empty input, and no-solution cases.
7. Give time and auxiliary-space complexity for each approach.
8. Recognize how this problem grows into sliding windows, fast/slow pointers, interval algorithms, selection, and comparison sorting.

1. The motivating problem: Target Pair

A monitoring service records integer changes in resource usage. Given the changes and an alert target, find two different observations whose values add to the target.

Base problem

Given an integer array `a` and integer target `T`, return indices `i` and `j` such that:

`i != j` and `a[i] + a[j] == T`.

If no such pair exists, return `(-1, -1)`.

First example

TEXT

```
a = [11, 2, 7, 15, 3, 6]
T = 9
```

One valid answer: `(1, 2)`, because `a[1] + a[2] = 2 + 7 = 9`.

Think before coding

Answer these questions in writing:

1. Are we returning values or original indices?
2. Can the same array position be used twice?
3. May the algorithm change the input order?
4. If several pairs work, must we return all of them?
5. What should happen when there is no pair?
6. Can values or the target be negative?

Specification habit: Many “algorithm errors” are actually failures to decide what the output means.

Edge-case gallery

TEXT

<code>[4], T = 8</code>	No answer: one position cannot be reused.
<code>[4, 4], T = 8</code>	Valid: the equal values occupy different positions.
<code>[-3, 1, 7, 10], T = 4</code>	Valid: <code>-3 + 7 = 4</code> .
<code>[], T = 0</code>	No answer.
<code>[2, 5, 9], T = 20</code>	No answer.

Checkpoint 1

For `a = [5, 1, 5, 0, -2, 7]` and `T = 10`:

- Give one valid index pair.
- Explain why using index `0` twice is invalid.
- Would sorting the array without remembering indices still satisfy the specification?

2. Approach A: Compare every pair

The most direct algorithm tries every pair `(i, j)` with `i < j`.

CPP

```
pair<int, int> targetPairBruteForce(const vector<int>& a, int target) {
    const int n = static_cast<int>(a.size());
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (a[i] + a[j] == target) {
                return {i, j};
            }
        }
    }
    return {-1, -1};
}
```

Walkthrough

For $a = [11, 2, 7, 15, 3, 6]$, $T = 9$:

Comparison	Sum	Decision
11 and 2	13	Continue
11 and 7	18	Continue
11 and 15	26	Continue
11 and 3	14	Continue
11 and 6	17	Continue
2 and 7	9	Return their indices

Correctness argument

Every pair of distinct indices appears exactly once with the smaller index first. Therefore, if a valid pair exists, the algorithm eventually checks it. It returns only after verifying the required sum.

Complexity

The number of pairs is:

$n(n-1)/2$, which is $\Theta(n^2)$.

- Time: $\Theta(n^2)$ in the worst case.
- Auxiliary space: $\Theta(1)$.
- Input order: preserved.

Why keep brute force? It is easy to trust, useful for small input, and excellent as an oracle for testing optimized solutions.

Checkpoint 2

1. How many pairs are checked when $n = 1000$ and no answer exists?
2. What input causes the function to return after only one comparison?
3. Why does starting j at $i + 1$ solve two separate correctness problems?

3. What does sorting buy us?

The brute-force method knows nothing about values it has not checked. Ordering changes that.

Sort the example:

TEXT

```
Original:  [11, 2, 7, 15, 3, 6]
Sorted:    [ 2, 3, 6,  7,11,15]
```

If the smallest and largest values sum to too much, then the largest value cannot pair successfully with the smallest—and pairing it with anything larger only makes the sum larger. We may eliminate it.

This is the key benefit:

Order turns observations into deductions.

But sorting also has costs:

- It usually takes $\Theta(n \log n)$ comparisons.
- It may modify the input.
- Original positions are lost unless we store them.
- Temporal or positional adjacency may be destroyed.

Preserve identity while sorting

Store `(value, originalIndex)` pairs:

CPP

```
vector<pair<int, int>> indexed;
for (int i = 0; i < static_cast<int>(a.size()); ++i) {
    indexed.push_back({a[i], i});
}

sort(indexed.begin(), indexed.end());
```

For the example:

TEXT

```
[(2,1), (3,4), (6,5), (7,2), (11,0), (15,3)]
```

The values are ordered, but each value still carries its original identity.

Checkpoint 3

Sort these pairs lexicographically:

TEXT

```
[(5,0), (1,1), (5,2), (0,3), (-2,4), (7,5)]
```

What information would be lost if you stored only the sorted values?

4. Approach B: Sort and binary-search for complements

For every value `x`, search for the complement `target - x`.

CPP

```
pair<int, int> targetPairBinarySearch(const vector<int>& a, int target) {
    vector<pair<int, int>> v;
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        v.push_back({a[i], i});
    }
    sort(v.begin(), v.end());

    for (int i = 0; i < static_cast<int>(v.size()); ++i) {
        const int need = target - v[i].first;
        auto first = v.begin() + i + 1; // search only later positions
        auto it = lower_bound(first, v.end(), make_pair(need, -1));
        if (it != v.end() && it->first == need) {
            return {v[i].second, it->second};
        }
    }
    return {-1, -1};
}
```

Why search only after *i*?

Searching the suffix ensures:

1. We do not reuse the same sorted position.
2. We do not repeat pairs already considered.

Binary-search invariant

At every step, if the target exists, it remains inside the active half-open range `[low, high)`.

TEXT

If middle value < target: discard middle and everything left of it.
Otherwise: keep middle and discard everything right of it.

This particular rule finds the first position whose value is at least the target: a lower bound.

Walkthrough

For sorted values `[2, 3, 6, 7, 11, 15]` and `T = 9`:

1. Choose 2; need 7.
2. Binary-search the suffix `[3, 6, 7, 11, 15]`.
3. The search eliminates half of the remaining range at every step.
4. Find 7, then return original indices 1 and 2.

Complexity

- Sorting: $\Theta(n \log n)$.
- *n* binary searches: $\Theta(n \log n)$.
- Total: $\Theta(n \log n)$.
- Extra space in this implementation: $\Theta(n)$ for indexed values.

Guided exercise A: lower bound by hand

On `[2, 3, 6, 7, 11, 15]`, trace lower-bound search for:

1. 7
2. 8
3. 1
4. 20

For each query, give the returned insertion index.

5. Approach C: Sort and use two pointers

Binary search works, but it repeatedly searches similar ranges. Two pointers exploit the sum's monotonic behavior directly.

C++

```
pair<int, int> targetPairTwoPointers(const vector<int>& a, int target) {
    vector<pair<int, int>> v;
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        v.push_back({a[i], i});
    }
    sort(v.begin(), v.end());

    int left = 0;
    int right = static_cast<int>(v.size()) - 1;

    while (left < right) {
        long long sum = static_cast<long long>(v[left].first)
            + static_cast<long long>(v[right].first);
        if (sum == target) {
            return {v[left].second, v[right].second};
        }
        if (sum < target) {
            ++left;
        } else {
            --right;
        }
    }
    return {-1, -1};
}
```

Detailed trace

Target $T = 9$, sorted values $[2, 3, 6, 7, 11, 15]$:

Left	Right	Sum	Deduction	Move
2	15	17	Too large; 15 with any remaining left value is no better	Right leftward
2	11	13	Too large; eliminate 11	Right leftward
2	7	9	Target found	Stop

The elimination proof

Suppose $v[\text{left}] + v[\text{right}] < \text{target}$.

Because $v[\text{left}]$ is the smallest remaining value, pairing it with any index left of right produces a sum no larger than the current sum. None can reach the target. Therefore left cannot belong to a solution in the active range, so incrementing it is safe.

The symmetric argument justifies decrementing right when the sum is too large.

Two-pointer invariant

If an unexamined solution exists, both of its indices lie inside the current closed interval $[\text{left}, \text{right}]$.

Each move shrinks that interval without discarding a possible solution.

Complexity

- Sorting: $\Theta(n \log n)$.
- Pointer scan: $\Theta(n)$ because each pointer moves at most $n - 1$ times.
- Total: $\Theta(n \log n)$.
- Extra space here: $\Theta(n)$ to preserve indices.

Overflow detail: Use `long long` for the sum when two `int` values might overflow.

Guided exercise B: justify every move

Trace the two-pointer method on:

TEXT

```
a = [-8, -3, 1, 2, 4, 6, 10]
T = 5
```

For each pointer move, write the entire region of pairs that the move eliminates.

6. Approach D: Hash complements

Sorting is not the only way to avoid repeated work. A hash table remembers values already seen.

CPP

```
pair<int, int> targetPairHash(const vector<int>& a, int target) {
    unordered_map<int, int> seen;
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        const int need = target - a[i];
        auto it = seen.find(need);
        if (it != seen.end()) {
            return {it->second, i};
        }
        seen[a[i]] = i;
    }
    return {-1, -1};
}
```

Hash invariant

Before processing index `i`, `seen` contains values from indices strictly smaller than `i`. Therefore a found complement always comes from a different position.

Trade-off

- Expected time: $\Theta(n)$.
- Worst-case time: potentially $\Theta(n^2)$ with pathological hashing.
- Auxiliary space: $\Theta(n)$.
- Original order: preserved.
- Sorted-order benefits: not obtained.

Method-selection table

Situation	Natural first choice	Reason
Tiny input	Brute force	Simplicity may dominate

Situation	Natural first choice	Reason
Already sorted input	Two pointers	Linear scan and constant extra space
Need original order and one query	Hash map	Expected linear time
Many ordered queries	Sort once	Preprocessing can be reused
Need all unique value-pairs	Sort + two pointers	Duplicate runs are explicit
Very limited memory	In-place sort + pointers	Avoid a linear-size hash table
Need worst-case guarantees	Sort + pointers	Deterministic $O(n \log n)$ is straightforward

7. Variation: return all unique value-pairs

Now return every distinct value-pair (x, y) with $x \leq y$ and $x + y = T$.

TEXT

```
a = [1, 1, 2, 2, 3, 3, 4, 4]
T = 5
```

Output: [(1, 4), (2, 3)]

The index pairs are numerous, but there are only two unique value-pairs.

CPP

```
vector<pair<int, int>> allUniquePairs(vector<int> a, int target) {
    sort(a.begin(), a.end());
    vector<pair<int, int>> answer;
    int left = 0;
    int right = static_cast<int>(a.size()) - 1;

    while (left < right) {
        long long sum = static_cast<long long>(a[left]) + a[right];
        if (sum < target) {
            ++left;
        } else if (sum > target) {
            --right;
        } else {
            int x = a[left];
            int y = a[right];
            answer.push_back({x, y});
            while (left < right && a[left] == x) ++left;
            while (left < right && a[right] == y) --right;
        }
    }
    return answer;
}
```

Why skip duplicates only after recording a pair?

Ordinary pointer movement is enough while the sum is wrong. Once a valid pair is recorded, every copy of the same left or right value would reproduce the same value-pair. Skipping the entire equal run prevents duplicate output.

Independent exercise 1

Trace `allUniquePairs` on:

TEXT

```
a = [-2, -2, 0, 0, 2, 2, 4, 4]
T = 2
```

Give the output and the pointer positions after each recorded pair.

Independent exercise 2

Modify the specification to count index-pairs, not unique value-pairs. For example, if there are three copies of 2 and four copies of 5, how many (2, 5) index-pairs exist? Describe how a sorted two-pointer algorithm should count whole duplicate blocks.

8. Pattern previews: similar-looking pointers, different invariants

Month 1 will develop three related but distinct patterns.

Opposite-direction two pointers

Typical setting: sorted values or a monotone relationship.

TEXT

```
left  --->                <--- right
```

Applications:

- Pair or triplet sums
- Closest sum
- Container-style optimization
- Palindrome checking
- Merging or partitioning ordered data

Invariant: pointer movement eliminates a provably impossible region.

Fast and slow pointers

Typical setting: one pointer explores while another marks processed structure.

TEXT

```
slow --->  
fast ----->
```

Array example: remove duplicates from a sorted array in place.

CPP

```
int removeDuplicates(vector<int>& a) {  
    if (a.empty()) return 0;  
    int slow = 1;  
    for (int fast = 1; fast < static_cast<int>(a.size()); ++fast) {  
        if (a[fast] != a[slow - 1]) {  
            a[slow] = a[fast];  
            ++slow;  
        }  
    }  
    return slow;  
}
```

Invariant: $a[0 \dots \text{slow}]$ contains the unique values found in $a[0 \dots \text{fast}]$.

Later applications include linked-list middle finding and cycle detection.

Sliding windows

Typical setting: a contiguous interval in the original order.

TEXT

```
[ outside ][ left ... active window ... right ][ outside ]
```

Example: maximum sum of any three consecutive values.

TEXT

```
a = [4, -1, 2, 10, -3, 5]
```

Windows: $4-1+2=5$, $-1+2+10=11$, $2+10-3=9$, $10-3+5=12$

Answer: 12

Sorting this array would destroy the meaning of “consecutive.”

Critical distinction: Two pointers often exploit sorted order. Sliding windows preserve positional order. Fast/slow pointers maintain a processed-region or relative-speed invariant.

Checkpoint 4: name the pattern

Choose the most natural pattern and justify it:

1. Determine whether a sorted array contains two values summing to 100.
2. Remove repeated values from a sorted array in place.
3. Find the longest contiguous substring containing at most two distinct characters.
4. Find the middle node of a linked list.
5. Merge two sorted arrays.

9. Sorting: the undergraduate map

Sorting is both a family of algorithms and a design tool.

Comparison-sorting landscape

Algorithm	Best	Average	Worst	Stable?	In-place?	Main lesson
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Yes	Adaptive; good for small/nearly sorted data
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Usually no	Yes	Few swaps, but comparisons stay quadratic
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	Usually no	Divide, solve, and merge ordered halves
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No	Mostly	Partitioning and pivot quality
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Yes	Strong worst-case bound with small extra space

Beyond comparison sorting

- Counting sort uses a bounded integer key range.
- Radix sort processes digits or key components.
- Bucket methods use assumptions about distribution.

They can beat $O(n \log n)$ because they obtain information without relying only on pairwise comparisons.

Why stability matters

Suppose records are already ordered by name:

TEXT

(Asha, 82), (Dev, 91), (Mina, 82), (Zoya, 91)

A stable sort by score preserves name order among equal scores:

TEXT

(Asha, 82), (Mina, 82), (Dev, 91), (Zoya, 91)

Stability makes successive sorts by different keys predictable.

Sorting as preprocessing

Sorting supports:

- Binary search and repeated queries
- Duplicate detection and grouping
- Two-pointer scans
- Interval merging
- Sweep-line algorithms
- Greedy scheduling
- Median and quantile reasoning
- Canonical representations for comparison
- External sorting of data larger than memory

GATE concept check

Mark each statement true or false and justify it:

1. Every $O(n \log n)$ sorting algorithm is stable.
2. Insertion sort can run in linear time on an already sorted array.
3. Comparison sorting can guarantee $O(n \log n)$ time for arbitrary distinct keys.
4. Merge sort is naturally suited to linked lists.
5. Sorting is harmless when a problem depends on original adjacency.
6. Heap sort is stable by default.

10. Practice set

Do not read the solution appendix until you have written an invariant and attempted an implementation.

Problem 1 · Direct recognition

Given a sorted array and target T , return whether any two distinct values sum to T using $O(n)$ time and $O(1)$ auxiliary space.

TEXT

Input: $[-7, -1, 2, 4, 8, 11]$, $T = 7$
Output: true

Write:

- The algorithm
- Its invariant
- Why each pointer move is safe
- Time and space complexity

Problem 2 · Implementation detail

Given a sorted array, remove duplicates in place and return the number of unique values. The first returned-count positions must contain those values in sorted order.

TEXT

Input: [1, 1, 1, 2, 2, 4, 7, 7]
 Output length: 4
 Modified prefix: [1, 2, 4, 7]

Do this in $O(n)$ time and $O(1)$ auxiliary space.

Problem 3 · Transfer

Given an unsorted array, return the pair whose sum is closest to T . If two pairs are equally close, return the lexicographically smaller value-pair.

TEXT

Input: [10, 22, 28, 29, 30, 40], $T = 54$
 Output: (22, 30)

Explain why sorting and two pointers still work even though equality may never occur.

Problem 4 · Constraint changes the pattern

Given an array and integers T and k , determine whether there are indices $i \neq j$ such that:

$a[i] + a[j] == T$ and $\text{abs}(i - j) \leq k$.

Explain why freely sorting the input is dangerous. Design a method that scans in original order while remembering only the previous k positions.

Problem 5 · Many queries

A fixed array of one million integers receives 100,000 target-pair queries. Compare:

- Repeating brute force
- Building a hash set per query
- Sorting once and scanning with two pointers per query
- Precomputing every possible pair sum

Discuss time, space, and when each method becomes unreasonable. There is no single universally best answer.

Problem 6 · Test design

Create at least eight tests for a target-pair implementation. Your suite must include:

- Empty input
- One element
- Duplicate values that do form a pair
- A tempting self-pair that must be rejected
- Negative values

- No solution
- Multiple solutions
- Values large enough to make `int + int` overflow

11. Interview follow-ups

Practice answering these aloud:

1. Why is the two-pointer scan linear after sorting?
2. If hashing is expected $O(n)$, why might you still choose sorting?
3. What does stable sorting mean, and does pair sum require stability?
4. How would the answer change if the input were already sorted?
5. How would you return all index-pairs rather than all unique value-pairs?
6. Can ordinary sliding window solve pair sum? Why or why not?
7. What information is destroyed by sorting?
8. What is your loop invariant?

12. Exit ticket

Complete without notes:

1. State the two-pointer invariant for target pair.
2. Explain in one sentence why `sum < target` allows `left` to move right.
3. Name one situation where hashing is preferable.
4. Name one situation where sorting is preferable.
5. Explain the difference between opposite-direction two pointers, fast/slow pointers, and a sliding window.
6. Give one reason to keep a brute-force implementation.

Self-rating

Rate each from 1 (not yet) to 4 (independent):

Skill	1	2	3	4
I can specify the problem precisely	☐	☐	☐	☐
I can trace binary search without boundary guessing	☐	☐	☐	☐
I can justify every two-pointer move	☐	☐	☐	☐
I can handle duplicate values correctly	☐	☐	☐	☐
I can choose between sorting and hashing	☐	☐	☐	☐
I can state time and auxiliary-space complexity	☐	☐	☐	☐

Solution appendix

Stop here until you have attempted the checkpoints and practice set.

Checkpoint answers

Checkpoint 1

A valid pair is indices $(0, 2)$ because the two different positions both hold 5. Reusing index 0 would violate $i \neq j$. Sorting values alone loses the original indices, so it cannot satisfy an index-returning specification.

Checkpoint 2

For $n = 1000$, brute force checks $1000 \times 999 / 2 = 499,500$ pairs. An answer using indices 0 and 1 returns after one comparison. Starting j at $i + 1$ prevents self-pairing and prevents checking both (i, j) and (j, i) .

Checkpoint 3

TEXT

$[(-2, 4), (0, 3), (1, 1), (5, 0), (5, 2), (7, 5)]$

Storing only values loses the positions needed by the original problem.

Guided exercise A

Lower-bound insertion indices in $[2, 3, 6, 7, 11, 15]$ are: $7 \rightarrow 3$, $8 \rightarrow 4$, $1 \rightarrow 0$, and $20 \rightarrow 6$.

Guided exercise B

Start with $-8 + 10 = 2$, so eliminate -8 ; $-3 + 10 = 7$, so eliminate 10 ; $-3 + 6 = 3$, so eliminate -3 ; $1 + 6 = 7$, so eliminate 6 ; $1 + 4 = 5$, so the pair is $(1, 4)$.

Checkpoint 4

1. Opposite-direction two pointers.
2. Fast/slow pointers for in-place compaction.
3. Sliding window.
4. Fast/slow pointers moving at different rates.
5. Same-direction pointers, one into each input sequence.

GATE concept check

1. False: runtime does not determine stability.
2. True: insertion sort performs no shifts on sorted input.
3. False in the comparison model: arbitrary distinct keys require $\Omega(n \log n)$ comparisons in the worst case.
4. True: merging linked lists needs pointer relinking rather than an auxiliary array.
5. False: sorting destroys original adjacency.
6. False: ordinary heap sort may reverse equal-key elements.

Practice-set guidance

Problem 1

Use $left = 0$ and $right = n-1$. The active interval retains every possible solution. Move $left$ when the sum is small and $right$ when it is large. Each pointer moves at most $n-1$ times, giving $O(n)$ time and $O(1)$ space.

Problem 2

Let `slow` be the length of the unique prefix and let `fast` scan the input. Copy `a[fast]` only when it differs from the last value in the unique prefix. The invariant is that `a[0..slow)` contains exactly the unique values from the processed prefix.

Problem 3

Sort the values. Maintain the best absolute difference and best value-pair. If the sum is below `T`, only increasing the smaller value can move toward `T`; otherwise decrease the larger value. Check the tie-breaking rule whenever updating the best answer.

Problem 4

Sorting is unsafe because the constraint uses original index distance. Scan left to right while maintaining frequencies of the previous `k` values. Before inserting `a[i]`, test whether `T - a[i]` is present. Remove `a[i - k]` when it leaves the permitted window. Expected time is $O(n)$ and space is $O(k)$.

Problem 5

Brute force is approximately $O(qn^2)$ and is infeasible. Rebuilding a hash set is $O(qn)$ expected time. Sorting once costs $O(n \log n)$, followed by $O(qn)$ two-pointer work, with lower auxiliary memory per query. Precomputing pair sums costs up to $\Theta(n^2)$ time and space and is infeasible for one million inputs unless the value universe is very small and admits compression.

Problem 6

A useful suite includes `[]`, `[4]` with target 8, `[4, 4]` with target 8, `[4, 1]` with target 8, `[-3, 1, 7]` with target 4, a no-solution case, a case with several valid pairs, and values near `INT_MAX/INT_MIN`. For small random arrays, compare the optimized result with brute force.

Looking ahead

The next sessions will deepen the tools introduced here:

- Binary-search boundaries and first/last occurrence
- Binary search on a monotone answer
- Two-pointer families and three-sum reasoning
- Fast/slow in-place transformations
- Fixed and variable sliding windows
- Comparison-sorting implementations and proofs
- Stability, comparators, partitioning, selection, intervals, and sweep methods

Keep returning to the central question:

What structure does the input already have, what structure can we create, and what deductions does that structure permit?